

RECOVERING STRUCTURE FROM DAMAGED OR POORLY SCANNED DOCUMENTS

An End-to-End Pipeline Combining Optical Character Recognition, Synthetic Damage Modeling, and Confidence-Calibrated Language Model Reconstruction

A Technical Paper Submitted in Fulfilment of the Requirements of the
Advanced Machine Learning Internship (Track 2)
Problem ID: ML-T2-022 — Learn Depth Academy LLP (LearnDepth™)

AUTHOR PROFILE

Gyimah Ramsey Opoku

STUDENT ID	LD-1787404180111
PROGRAMME	BSc. Computer Science (Third Year)
DEPARTMENT	Department of Computational and Physical Sciences
COLLEGE	College of Science
INSTITUTION	Kwame Nkrumah University of Science and Technology (KNUST), Kumasi, Ghana
INTERNSHIP	Advanced Machine Learning Internship, Track 2 — Learn Depth Academy LLP
REPOSITORY	github.com/aimlin9/document-recovery-ml-internship
LINKEDIN	linkedin.com/in/ramsey-opoku-gyimah-79a4b72aa

This paper documents the design, implementation and evaluation of a five-stage pipeline that recovers text from torn and damaged document scans. It pairs Tesseract OCR and a reproducible synthetic-damage generator with a fine-tuned RoBERTa model that reconstructs damaged words. A confidence layer calibrated on a validation set flags uncertain reconstructions for human review. Every output is a schema-validated JSON document that records, word by word, what was observed and what was inferred. Evaluated end to end on 739 held-out documents, the system reduces character error rate from 4.92% to 3.00% relative to OCR alone.

Recovering Structure From Damaged or Poorly Scanned Documents: An End-to-End Pipeline Combining OCR, Synthetic Damage Modeling, and Confidence-Calibrated Language Model Reconstruction

Gyimah Ramsey Opoku

Student ID: LD-1787404180111 · BSc. Computer Science (Third Year)

Department of Computational and Physical Sciences, College of Science

Kwame Nkrumah University of Science and Technology (KNUST), Kumasi, Ghana

Advanced Machine Learning Internship, Track 2, Problem ID: ML-T2-022 — LearnDepth™

github.com/aimlin9/document-recovery-ml-internship

September 2026

Abstract

Physical and digitized archives frequently contain documents that are torn, water-damaged, faded, or otherwise degraded, and conventional Optical Character Recognition (OCR) systems produce systematically unreliable transcriptions of such material because their underlying models assume an intact page. This paper presents an end-to-end pipeline for recovering structure and content from damaged documents, developed across five sequential stages: (1) establishing a baseline characterization of OCR degradation under controlled synthetic damage, (2) building a damage-mask generator that produces reproducible torn and missing regions with known ground truth, (3) evaluating a pretrained masked-language-model reconstruction baseline, (4) fine-tuning that model on the reconstruction task and adding a calibrated confidence layer that flags unreliable reconstructions for human review, and (5) integrating all preceding components into a single schema-validated pipeline evaluated end-to-end on a 739-document held-out test set.

The final system reduces corpus-level character error rate from 4.92% (OCR alone) to 3.00% (full pipeline) and word error rate from 5.60% to 3.83%. Its uncertainty layer, calibrated purely on text-only data in Stage 4, transfers only partially to the real image-based evaluation of Stage 5: it recovers 65.76% of reconstruction errors at 84.30% precision, against a validation-set performance of 80.06% recall — a generalization gap that this paper treats as a substantive finding rather than a defect to be reconciled away. The paper documents the full methodology, every schema and implementation decision, the bugs discovered during development and how they were diagnosed and fixed, and the scope limitations of the current system, together with a concrete plan for future work.

Keywords: *document image restoration, optical character recognition, masked language modeling, RoBERTa, uncertainty calibration, JSON Schema, digital archiving*

Contents

1. Introduction.....	3
1.1 A Motivating Scenario.....	4
1.2 Contributions.....	4
2. Literature Review.....	4
3. Problem Statement and Objectives.....	6
4. System Overview.....	8
5. Methodology.....	10
5.1 Corpus Construction.....	10
5.2 Step 1 — Baseline OCR Under Synthetic Degradation.....	10
5.3 Step 2 — Damage Mask Generation and Classification.....	10
5.4 Step 3 — Pretrained Reconstruction Baseline.....	11
5.5 Step 4 — Fine-Tuning and Uncertainty Calibration.....	11
5.6 Step 5 — End-to-End Integration.....	13
5.7 Evaluation Metrics: Formal Definitions.....	14
5.8 Training Configuration Summary.....	15
6. System Architecture and the Integration Schema.....	17
7. Implementation Details and Engineering Challenges.....	18
8. Results.....	22
9. Discussion.....	25
10. Limitations and Scope.....	27
11. Future Work.....	28
12. Conclusion.....	29
13. References.....	30
Appendix A — Full JSON Integration Schema.....	31
Appendix B — Key Source Code Listings.....	35
Appendix C — Full Step 5 Aggregate Results.....	36
Appendix D — Reproducibility Manifest.....	37
Appendix E — Held-Out Manifest and Metric Computation.....	37
Appendix F — Glossary of Terms.....	39
Appendix G — Notation and Symbol Reference.....	39

List of Tables and Figures

Table 1. Step 4 — Validation vs. test-set uncertainty-flagging performance.....	22
Table 2. Step 5 — Full held-out evaluation results (n = 739).....	22
Table 3. Summary comparison across development stages.....	23
Figure 1. Step 5 error-rate reduction and uncertainty-flagging performance.....	23

1 INTRODUCTION

Archives, libraries, courts, universities, and government registries around the world hold enormous volumes of paper records that predate digital record-keeping, and a large share of that material has suffered some form of physical degradation: torn corners and margins, water and fire damage, insect and mould damage, ink fading, and mechanical wear from repeated handling. As institutions digitize these holdings, a naive pipeline of “scan, then run OCR” quietly discards information whenever a scanned page includes a degraded region: standard OCR engines are trained and tuned for reasonably intact print, and they respond to damage not by reporting uncertainty but by silently emitting whatever character sequence their language and shape models judge most likely, often with no visible signal to a downstream reader that a particular word is a guess.

This creates two compounding problems for any organization trying to build a trustworthy digital archive from damaged source material. First, transcription errors introduced by damage are indistinguishable, in the output stream, from transcription errors introduced by an intact page that is simply hard to read — a system that cannot tell the two apart cannot tell a human reviewer where to look. Second, even where the general shape of the problem is understood, there has been comparatively little openly documented, end-to-end tooling that combines damage-aware OCR, learned reconstruction of missing text, and calibrated uncertainty estimation into one pipeline with a clearly specified data contract between its stages.

This paper documents the design, implementation, and evaluation of such a pipeline, built over five sequential development stages as part of an

Advanced Machine Learning internship (Track 2, Problem ID ML-T2-022) at Learn Depth Academy LLP. The work follows a deliberately incremental methodology: rather than attempting an end-to-end system from the outset, each stage isolates and evaluates one component in controlled conditions before the next stage builds on it, and every stage's results were documented and independently verified before proceeding. This incrementalism is itself a methodological contribution of the project, and it surfaces repeatedly in this paper: several of the most useful findings (a multi-word reconstruction bug, a schema type mismatch, a generalization gap between calibration and deployment conditions) were only visible because each stage's assumptions were checked against direct evidence rather than trusted by construction.

The remainder of this paper is organized as follows. Section 2 surveys related work in OCR, masked language modeling, and confidence calibration. Section 3 states the problem and objectives precisely. Section 4 gives a system-level overview of the pipeline. Section 5 is the core of the paper: a stage-by-stage account of the methodology, decisions, and verification performed at each of the five development steps. Section 6 documents the JSON integration schema that ties the pipeline together as a formal data contract. Section 7 discusses implementation challenges and the bugs found and fixed during development. Section 8 presents the full quantitative results, culminating in the Step 5 evaluation across the entire 739-document held-out test set. Section 9 discusses what these results mean, including a generalization gap that the project treats as a legitimate finding. Section 10 states the system's scope and limitations plainly. Section 11 outlines future work, and Section 12 concludes.

1.1 A MOTIVATING SCENARIO

Concretely, consider an archivist at a university library digitizing a collection of decades-old correspondence and administrative records. Many pages have suffered edge tears, water spotting near the binding, or ink that has faded unevenly across the page. A naive digitization workflow scans each page and runs it through a standard OCR engine, then stores the resulting text as the searchable record. Wherever a tear removes part of a word, the OCR engine does not report a gap: it reports its best guess at a full word, indistinguishable in the output from a word it read perfectly. Months later, a researcher searching the digitized archive for a specific term has no way to know that some fraction of the words in front of them were never actually read from the page at all. This is precisely the failure mode this project is built to address: not by making OCR itself more accurate on damaged print, which is a hard problem in its own right, but by making the pipeline honest about which words are reconstructions and how much to trust each one, so that a researcher, or an automated downstream system, can make an informed decision about what to verify against the original scan before relying on it.

1.2 CONTRIBUTIONS

- An end-to-end, five-stage pipeline that combines damage-aware OCR baselining, synthetic damage simulation with known ground truth, pretrained and fine-tuned masked-language-model reconstruction, and validation-calibrated uncertainty flagging into a single schema-validated system (Sections 4–6).
- A documented generalization gap between text-only confidence calibration and real, image-based deployment conditions, reported as a substantive finding with the exact numbers on both sides (Section 9.1).

- A fully reproducible evaluation methodology in which every reported figure is traceable to a script, a corpus file hash, and a record count captured at run time, rather than to a remembered or assumed number (Section 7.3, Appendix D, Appendix E).
- A catalogue of real integration bugs, each with the diagnostic evidence used to find and fix it, offered as a case study in how end-to-end testing surfaces failure modes that isolated, component-level testing cannot (Section 7).

2 LITERATURE REVIEW

2.1 OPTICAL CHARACTER RECOGNITION

Tesseract, the OCR engine used throughout this project, is an open-source engine originally developed at Hewlett-Packard and later maintained by Google; its architecture and character-recognition approach are documented by Smith (2007) [1]. Tesseract, like most classical and modern OCR systems, performs best on print that closely resembles its training distribution: consistent contrast, unbroken glyph shapes, and a legible font. Under physical damage — torn regions, missing ink, heavy staining — glyph shapes are altered or removed entirely, and the engine's internal language model, tuned for correcting ordinary recognition ambiguity rather than compensating for missing pixels, produces low-confidence or simply wrong output without a reliable signal that damage, specifically, was the cause.

2.2 MASKED LANGUAGE MODELS FOR TEXT RECONSTRUCTION

The masked-language-modeling objective, introduced by Devlin et al. (2018) in BERT [2], trains a transformer to predict a word given its full bidirectional context. This objective is a natural fit for reconstructing a damaged or missing word from

the surrounding, undamaged text, since the model is trained exactly to do this. This project uses RoBERTa (Liu et al., 2019) [3], a re-tuned and more heavily trained variant of the BERT architecture, as its reconstruction backbone, first evaluating it in its pretrained form and later fine-tuning it directly on the reconstruction task.

2.3 POST-OCR ERROR CORRECTION

A growing line of work applies language models to correct OCR output after the fact rather than only to guide recognition itself. Recent evaluations of large language models for post-OCR correction of historical documents (2025) [5] report that such correction is not uniformly beneficial — improvements are inconsistent across document types and degradation levels, and correction can occasionally introduce new errors into text the OCR engine had transcribed correctly. This finding directly motivated one of the design choices in this project: reconstruction is deliberately restricted to words the pipeline has independently classified as damaged (Step 2), rather than being applied indiscriminately to every OCR output, precisely to avoid the failure mode of “fixing” text that was never broken.

2.4 CALIBRATION AND UNCERTAINTY ESTIMATION

A model's stated confidence is only useful if it reflects the model's actual accuracy. Guo et al. (2017) [4] showed that modern neural networks are frequently overconfident relative to their true accuracy and popularized Expected Calibration Error (ECE) as a standard metric for measuring the gap between stated confidence and observed accuracy across confidence bins. This project applies the same calibration methodology — binning predictions by confidence and comparing average confidence to observed accuracy within each bin — to the reconstruction model's own confidence scores, and reports the resulting ECE alongside a

validation-locked decision threshold for flagging uncertain reconstructions.

2.5 DATA CONTRACTS FOR MULTI-STAGE PIPELINES

Beyond the machine learning literature, this project draws on the practice, common in production data engineering, of specifying a formal schema (here, JSON Schema, draft 2020-12 [6]) as the contract between pipeline stages, so that each component's inputs and outputs are independently specifiable and testable rather than implicitly agreed upon through code alone.

2.6 SYNTHETIC DAMAGE SIMULATION AS A BENCHMARKING PRACTICE

Evaluating a document-recovery system directly on genuinely damaged historical material has a fundamental limitation: for most real damaged documents, the original, undamaged ground-truth text is unknown, which makes it impossible to measure reconstruction accuracy directly. Synthetic damage simulation — deliberately degrading a clean, known document and then measuring how well a system recovers the original — sidesteps this limitation and is a standard benchmarking practice across document image analysis, comparable to how synthetic noise, blur, and compression artifacts are used to benchmark general image restoration systems. This project follows that practice throughout Steps 1 through 5, and is explicit, at every stage, that measured accuracy is conditioned on the specific synthetic damage model used (a background-color-matched ribbon tear) and does not, by itself, establish how the system would perform against a different damage distribution (water staining, ink bleed, insect damage) without separate evaluation — a point revisited in Section 10.

2.7 SUMMARY AND POSITIONING

Taken together, the literature establishes three separate capabilities — OCR, masked-language-model reconstruction, and calibrated uncertainty estimation — each well studied on its own, but rarely combined into one openly documented pipeline with a formal, testable data contract between stages. This project's positioning is not to advance any one of these three techniques individually, but to combine them carefully, to be explicit about where the combination's confidence guarantees do and do not hold (Section 9.1), and to treat integration testing itself as a source of findings rather than a formality performed after the “real” research is done.

2.8 POSITIONING RELATIVE TO END-TO-END DOCUMENT AI SYSTEMS

A separate and much larger body of work, often grouped under the label “Document AI,” builds transformer models that jointly reason over a document's text, layout, and visual appearance, typically for tasks such as form-field extraction, table understanding, or document classification. These systems are trained to understand where information sits on a page and what role it plays, and they are, in general, far more capable than this project's pipeline at structured information extraction from well-formed documents. This project is deliberately narrower and, in exchange, deeper on one specific sub-problem: it does not attempt to understand a document's layout or semantic structure at all, and instead concentrates every design decision on a single question — when OCR text is missing or unreliable because of physical damage, specifically, how should the missing content be reconstructed, and how confident should a reader be in that reconstruction. A production archival system would plausibly combine a Document-AI-style layout model for structure with

a damage-aware reconstruction and confidence layer of the kind built here; the two are complementary rather than competing approaches, and this paper does not claim to substitute for the former.

2.9 WHY WORD-LEVEL RECONSTRUCTION RATHER THAN FULL-PASSAGE GENERATION

An alternative design, considered and deliberately not pursued, would use a general-purpose large language model to regenerate an entire damaged passage at once, conditioned on the surrounding intact text. This was rejected for two concrete reasons specific to the archival use case this project targets. First, passage-level generation makes it substantially harder to localize which specific words were changed and which were left untouched, which directly undermines the provenance guarantee (Section 6) that this project treats as non-negotiable: an archivist must be able to tell, per word, whether they are reading an original observation or a machine-generated guess. Second, a passage-level generator's fluency objective rewards a smooth-reading paragraph over a faithful one, and the post-OCR correction literature reviewed in Section 2.3 already documents this exact failure mode — a model confidently rewriting text that did not need rewriting. Constraining reconstruction to individually classified damaged words, each retaining its own confidence score and provenance tag, was chosen specifically to avoid that failure mode, at the acknowledged cost of not being able to repair damage larger than a single word (a limitation stated explicitly in Section 10).

3 PROBLEM STATEMENT AND OBJECTIVES

3.1 PROBLEM STATEMENT

Given a scanned page image containing torn, missing, or otherwise damaged regions, produce a

best-effort reconstruction of the page's original text, together with a per-word estimate of how much that reconstruction should be trusted, in a structured, machine-readable form suitable for downstream archival and review workflows.

3.2 OBJECTIVES

- Characterize how much OCR accuracy degrades under controlled, reproducible synthetic damage, to establish a baseline against which any reconstruction method can be measured (Step 1).
- Build a damage simulation and per-word damage classification component that produces documents with known ground truth for benchmarking, while being explicit that this is a benchmarking convenience rather than a real damage detector (Step 2).
- Establish whether an off-the-shelf pretrained language model can reconstruct damaged words without any task-specific training, as a lower-bound baseline (Step 3).
- Improve reconstruction accuracy by fine-tuning the language model directly on the task, and attach a calibrated confidence score to every reconstruction so unreliable output can be flagged for human review rather than silently trusted (Step 4).
- Integrate OCR, damage classification, reconstruction, and confidence scoring into a single, schema-validated, end-to-end pipeline, and evaluate it as a whole system on a realistic, image-based, held-out test set (Step 5).

A sixth, cross-cutting objective governed how the first five were pursued: every quantitative claim in this paper is backed by a script that was actually run, on a corpus whose size and identity is fixed by a file hash recorded at run time, rather than by a remembered or assumed figure. Section 7.3 documents a case where this discipline caught a real discrepancy before it reached this paper.

3.3 SUCCESS CRITERIA

Two levels of success criteria were used to judge whether each development stage, and the project as a whole, met its goals.

Primary Success Indicators

- A working, schema-validated, end-to-end pipeline that runs without error across the full 739-document held-out test set (achieved: 739/739, zero errors, Section 8.2).
- A measurable reduction in corpus-level CER and WER relative to OCR alone (achieved: CER 4.92%→3.00%, WER 5.60%→3.83%, Section 8.2).
- A confidence-flagging layer whose validation-set calibration is documented with a stated operational goal, rather than an arbitrary threshold (achieved: 80.06% error recall at the locked threshold, Section 5.5.3).

Supporting Indicators

- Every quantitative claim traceable to a script and a run, not a remembered figure (Section 7.3).
- Every bug found during development documented with its diagnostic evidence, not just its fix (Section 7).
- Scope limitations stated explicitly rather than discovered by a reader (Section 10).

3.4 NON-GOALS

Stating what this project does not attempt is as important to a precise problem statement as stating what it does. The following are explicitly out of scope for the system described in this paper, not because they are unimportant, but because attempting them would have diluted the depth with which the stated objectives above were pursued and verified.

- Detecting damage location in a genuinely unknown scanned document. This project's damage classification (Section 4.2) is only

possible because the pipeline itself generated the damage mask; building a general damage-region detector is explicitly deferred to future work (Section 11).

- Reconstructing damage spanning more than one word. The architecture reconstructs one word position at a time (Section 4.3); multi-word or whole-line damage is out of scope for the current system (Section 10.2).
- Supporting languages other than English, or document types other than typeset literary prose. The corpus, OCR configuration, and reconstruction model are all specific to English literary text (Section 5.1, Section 10.5).
- Providing a production-ready service. The pipeline is a research and evaluation artifact, run as an offline batch process; it is not packaged as a deployable service with a request interface, queueing, or persistent storage (Section 4.7).

4 SYSTEM OVERVIEW

At a high level, the completed pipeline takes a single scanned page image and produces one JSON document conforming to the schema in Section 6. Internally, it proceeds through four functional stages for every page:

4.1 OCR

Tesseract is run on the (possibly damaged) page image, producing a raw transcription with a bounding box and a native confidence score for each detected word.

4.2 Damage Classification

Each expected word position (from the known, undamaged word layout, in benchmark mode) is compared against the pixel-level damage mask, and classified as intact, partially damaged, or missing,

based on what fraction of its bounding box overlaps the mask.

4.3 Reconstruction

Every word classified as partially damaged or missing is passed to the fine-tuned RoBERTa model, which iteratively fills in one, two, or three masked subword tokens and returns the highest-confidence single-word candidate, together with a geometric-mean confidence score over its constituent subword probabilities.

4.4 Confidence Flagging and Stitching

Each reconstructed word's confidence is compared against a threshold locked during Step 4's validation-only calibration; words at or below the threshold are flagged uncertain for human review. Every word's final text — either the OCR text (for intact words) or the reconstructed text (for damaged words) — is stitched into the page's final output text.

Section 6 and Appendix A specify this pipeline precisely: the schema is not merely documentation but an executable contract, checked by the jsonschema validator against every document the pipeline produces.

4.5 DESIGN PRINCIPLES

Three principles, stated here explicitly because they shaped every decision documented in Section 5, governed how this pipeline was built rather than only what it does:

- Verify before trusting. Every claim in this paper — a corpus size, a schema rule, a bug fix — was checked against direct evidence (a hash, a rendered PDF page, a deliberately broken test case) rather than accepted because it seemed plausible.
- Isolate before integrating. Each of Steps 1 through 4 evaluates one component under controlled, text-only or single-variable conditions before Step 5 combines them, so that a failure in the integrated system

can be traced to a specific stage rather than diagnosed from scratch.

- Report findings, not just results. A generalization gap (Section 9.1), a statistically undetectable improvement (Section 5.5.3), and an unresolved scope limitation (Section 10) are each reported in this paper with the same care as a positive result, because a technical report that only reports successes is less useful to whoever builds on it next.

4.6 DATA FLOW EXAMPLE

To make the four stages concrete, consider a single word position on a synthetically damaged page whose ground-truth text is “solutions.” OCR (4.1) either fails to detect the position at all or returns a low-confidence, garbled token, since the region is physically torn. Damage classification (4.2) compares this word's known bounding box against the pipeline's own damage mask and finds that more than 90% of the box overlaps the tear, so the word is classified missing. Reconstruction (4.3) masks this position and asks the fine-tuned RoBERTa model to fill it in; the model searches one-, two-, and three-subword-token candidates and, after the single-word-boundary fix described in Section 5.6.3, returns “solutions” (or a plausible near-miss) together with a geometric-mean confidence score. Confidence flagging (4.4) compares that score against the locked threshold (0.5442): if the score falls at or below the threshold, the word is marked uncertain in the final JSON document, so that a human reviewer's attention is directed to exactly this word rather than to the page as a whole. The word's `final_text`, `reconstruction_confidence`, `uncertain` flag, and `provenance: "reconstructed"` are then written into the document object described in Section 6, while every intact word on the same page carries `provenance: "ocr"` and null reconstruction fields, giving a reviewer a complete, word-by-word account of what was observed versus inferred.

4.7 DEPLOYMENT TOPOLOGY

As built, the pipeline runs as an offline, page-at-a-time batch process: it is invoked once per scanned image and produces one JSON document as output, with no persistent server component, no request queue, and no notion of a multi-page document beyond independently processing each page. This is a deliberate simplification appropriate to a research and evaluation context. A production deployment integrating this pipeline into an archive's ingestion workflow would additionally need a queue or job scheduler to process a backlog of scans, a persistent store for the resulting JSON documents (rather than one file per page on local disk), and a review interface through which an archivist can see flagged words in the context of the original page image — none of which this project builds, but all of which are compatible with the JSON schema (Section 6) as the interface boundary between this pipeline and whatever system consumes its output.

5 METHODOLOGY

5.1 CORPUS CONSTRUCTION

The underlying text corpus consists of individual sentences drawn from public-domain literary sources (including, among others, Jane Austen's *Pride and Prejudice*), stored one record per line in `sentence_corpus.jsonl` with fields `sentence`, `source`, and `split` (train / validation / test). Splitting at the sentence level, prior to any masking or damage simulation, ensures that no sentence used to fine-tune the reconstruction model in Step 4 also appears in the held-out test set evaluated in Step 5.

A dedicated diagnostic script, `check_corpus.py`, computes the SHA-256 hash of the corpus file and counts records by split directly from disk at run time. This was used specifically to resolve a discrepancy, described in Section 7.3, between a remembered test-set size of 757 sentences and the actual, verified figure, which is 739 test sentences — the figure used consistently throughout Steps 4 and 5 of this paper.

5.2 STEP 1 — BASELINE OCR UNDER SYNTHETIC DEGRADATION

Objective

Establish how Tesseract's OCR accuracy degrades as a function of synthetic damage severity, before any reconstruction component exists, so that later stages have a documented baseline to improve upon.

Method

Clean page images were rendered from corpus sentences and then degraded using a parameterized synthetic damage generator (`torn_regions.py`) capable of producing ribbon-shaped tears of varying size and position, with the surrounding background color estimated and used to fill the torn region so that the damage is visually consistent with the page rather than an obvious synthetic artifact. Tesseract was run on both the clean and the damaged images, and OCR accuracy (character- and word-level) was compared across a range of damage severities.

Outcome

OCR accuracy degraded predictably and substantially as damage severity increased, confirming the core premise of the project: an intact-page OCR engine has no mechanism for compensating for missing or altered pixels, and its output becomes systematically less trustworthy, without necessarily signalling this fact through its own native confidence scores. This baseline motivated the decision to build a dedicated damage-aware pipeline rather than relying on OCR confidence alone as a reliability signal.

5.3 STEP 2 — DAMAGE MASK GENERATION AND PER-WORD DAMAGE CLASSIFICATION

Objective

Produce reproducible synthetic damage with known ground truth, and classify each word's damage status against that ground truth, so that later reconstruction and evaluation stages have a well-defined notion of “which words are damaged, and how much.”

Method

torn_regions.py generates a binary damage mask over the page (generate_ribbon_tear), seeded for reproducibility, and applies it to a clean image (apply_tear) using an estimated background color (estimate_background_color) so that the damaged region blends into the page rather than appearing as an obviously synthetic patch. Separately, word_targeted_masks.py computes the bounding box of every word on the clean, undamaged image (get_word_boxes), which serves as ground truth for word position. reconstruction.py's classify_all_words function then computes, for every word, the fraction of its bounding box that overlaps the damage mask, and classifies the word as:

- Intact — less than 5% of the word's bounding box overlaps the damage mask.
- Partial — between 5% and 90% overlap.
- Missing — more than 90% overlap.

Scope Note

This classification is only possible because the pipeline itself generated the damage mask and therefore knows, with pixel-level precision, exactly where the damage is. This is explicitly documented, in both the codebase and the JSON schema (Section 6), as a benchmarking convenience rather than a general damage-detection capability: a real deployment on a genuinely damaged scan, with no known mask and no clean reference image, would require a separate damage-region detector that this project does not build. This limitation is treated as a stated scope boundary, not a hidden gap, and is revisited in Section 10.

5.4 STEP 3 — PRETRAINED RECONSTRUCTION BASELINE

Objective

Establish a lower-bound reconstruction baseline using an off-the-shelf pretrained RoBERTa model, with no task-specific fine-tuning, to quantify how much benefit fine-tuning (Step 4) subsequently provides.

Method

For each word classified as damaged in Step 2, its position in the sentence was masked and the pretrained roberta-base model was asked to fill in the missing token(s) using its standard masked-language-modeling head. Reconstruction accuracy was measured against the known ground-truth word.

Outcome

The pretrained model produced plausible, syntactically valid completions in many cases, but frequently selected a word that was locally fluent rather than the specific word originally present — for example, reconstructing “convenient” where the ground truth was a different, contextually specific word. This is expected: a pretrained model optimizes for general linguistic plausibility, not for matching one particular missing word, and it establishes precisely the gap that fine-tuning in Step 4 is designed to close.

5.5 STEP 4 — FINE-TUNING AND UNCERTAINTY CALIBRATION

This step comprised two connected pieces of work: improving raw reconstruction accuracy through fine-tuning, and building a principled confidence layer so that the system's own uncertainty is trustworthy rather than merely present.

5.5.1 Fine-Tuning and Addressing Overfitting

The pretrained RoBERTa model was fine-tuned on masked-word reconstruction examples built directly from the training split of the corpus (`build_masked_examples.py`), each example masking one word in a sentence and using the true word as the training target. An initial fine-tuning run showed signs of overfitting: training loss continued to decrease while validation performance plateaued relative to the best intermediate checkpoint. Two corrective actions were taken:

- Root-cause diagnosis over guesswork: rather than immediately assuming standard causes, the model's tokenizer was directly checked for a version mismatch between the transformers and tokenizers libraries, confirmed to be corrupting label alignment during training, and fixed by pinning compatible versions (`transformers==4.44.2`, `tokenizers==0.19.1`).
- `load_best_model_at_end=True`: the training configuration was changed to track validation loss across checkpoints and automatically restore the best-performing checkpoint at the end of training, rather than simply using the final epoch's weights.

Retraining with this setting produced a clear improvement: validation masked-language-modeling loss fell from 2.2367 to 1.6615, and whole-word reconstruction accuracy improved from 48.17%/26.90% to 48.65%/27.03% (overall / content-word accuracy, respectively, on the text-only masked evaluation set).

5.5.2 Confidence Scoring Methodology

Following explicit design guidance to avoid penalizing multi-subword-token words for having more pieces, reconstruction confidence is computed as the geometric mean of the per-subword-token probabilities the model assigned to its own chosen tokens while decoding a word:

```
confidence = exp( (1/n) * Σ log p_i ), for i = 1..n subword tokens
```

This is mathematically equivalent to averaging the tokens' log-probabilities and exponentiating the result, and it ensures that a correctly reconstructed two- or three-subword word is not automatically assigned a lower confidence purely because it required more decoding steps than a single-subword word.

5.5.3 Calibration Methodology

The confidence-correctness relationship was established, deliberately, on the validation set only, before any threshold was chosen, following this sequence:

- Confidence scores were grouped into ten equal-width bins, and average confidence was compared against actual accuracy within each bin.
- Expected Calibration Error (ECE) was computed as the weighted average, across bins, of the absolute gap between average confidence and observed accuracy.
- The model was confirmed to be overconfident in every bin, including the top bin — 96.9% average confidence against 94.9% observed accuracy in the highest-confidence bin, with an overall ECE of 0.0704.
- A decision threshold was chosen, using only the validation set, to satisfy a stated operational goal: catching approximately 80% of wrong reconstructions. The threshold selected was 0.5442, achieving 80.06% error recall and 79.61% precision at a 53.45% flag rate on the validation set.

This threshold was then locked and evaluated exactly once, without further adjustment, on the untouched 1,478-example text-only test set, where it achieved 77.21% error recall, 78.98% precision, and a 50.20% flag rate. Because a small headline point-difference between models can be noise rather than a real effect, a paired bootstrap (10,000 resamples) was run to quantify the reliability of the 0.48-point Top-1 accuracy improvement attributed to fine-tuning plus calibration; the result was +0.47 percentage points, 95% CI [-3.04, +4.06] — an interval that includes zero, meaning the true effect at this sample size is not statistically distinguishable from no improvement, though it remains compatible with an improvement as large as roughly four points. This paper reports that result exactly as it is: not “no effect,” but “no effect that this sample size can detect.”

Beyond recall and precision, the confidence layer's practical value to a human reviewer is best expressed by the unflagged error rate: of the reconstructions the system does not flag for review, only about 23.5% are wrong, compared with a 51.35% error rate across all reconstructions before any filtering — meaning an archivist who only reviews flagged words, and otherwise trusts the unflagged output, is working against a substantially cleaner stream of text than the raw reconstruction output alone would provide.

5.6 STEP 5 — END-TO-END INTEGRATION

Objective

Combine every prior stage into a single pipeline that produces one schema-validated JSON document per page, and evaluate that complete pipeline, as a system, on a realistic, image-based, held-out test set — not merely on the isolated text-only conditions under which each component was separately developed.

5.6.1 Designing the Integration Contract

Before writing the stitching code, a formal JSON Schema (draft 2020-12) was written to specify exactly what each pipeline component is responsible for producing, so that “the final stage will not invent fields that no component can support yet.” The schema is described fully in Section 6 and reproduced in Appendix A. It was validated in both directions: a fully populated example instance was confirmed to pass validation, and a deliberately broken instance (an intact word incorrectly carrying provenance: “reconstructed”) was confirmed to be rejected, so that the schema's rules were tested empirically rather than only described in prose.

5.6.2 The Stitching Pipeline

`build_document_json.py` orchestrates the full per-document pipeline: it loads the fine-tuned model (falling back, with a loud warning, to the pretrained model if the fine-tuned weights are not found locally, since the locked confidence threshold is calibrated specifically against the fine-tuned model's confidence distribution); runs OCR on the damaged image; matches OCR-detected words to their expected ground-truth positions using Intersection-over-Union bounding-box matching; and reconstructs every non-intact word using iterative masked decoding that tries one, two, and three mask tokens and keeps whichever candidate has the highest geometric-mean confidence.

5.6.3 A Real Bug, Found and Fixed

The first real run of the stitching pipeline surfaced a genuine defect: for a partially torn word whose ground truth was “solutions;”, the reconstruction step returned “choices, and” — a fluent, high-confidence three-subword-token decode that was nevertheless not a single word, because RoBERTa's subword vocabulary does

not respect word boundaries, and a three-mask-token search can and did find a high-confidence sequence spanning a word plus punctuation plus the start of the next word. The fix was to reject any decoded candidate containing a space before comparing confidences:

```
if len(predicted_ids) != num_mask_tokens:
    continue
word = tokenizer.decode(predicted_ids).strip()
if " " in word:
    continue # spans a word boundary - not a valid single-word candidate
confidence = math.exp(sum(log_probs) / len(log_probs))
```

Re-running the pipeline after this fix produced “choices;” for the same word, with confidence correctly falling from 0.193 to 0.1069 once the higher-confidence but invalid candidate was excluded from consideration. This episode is documented in Section 7 as a case study in how integration testing surfaces failure modes that unit-level, text-only evaluation cannot.

5.6.4 Full Held-Out Evaluation

`run_held_out_evaluation.py` runs the complete image-based pipeline across every sentence in the test split — 739 documents, each with exactly one deliberately targeted damaged word (selected via a per-document seeded random choice and `word_targeted_masks.build_targeted_mask`, rather than a whole-image random tear, specifically so that every document contributes exactly one reconstruction attempt rather than risking a tear that misses text entirely). The script:

- Hashes the corpus file and records the exact record count read at run time into an immutable `held_out_manifest.json`, rather than hardcoding a count anywhere in the code.
- Computes corpus-level (pooled, not per-document-averaged) CER and WER using the `jiwer` library's list-based API.
- Writes one JSON file per document (`results/held_out_documents/test_0000.json` through `test_0738.json`), an incremental results file flushed after every document, and one aggregate summary report.

A 20-document smoke test was run first, following the project's established practice of validating a pipeline at small scale before committing to a full run; it surfaced a schema/code bug (Section 7.4) that was fixed before the full run proceeded. The full 739-document run then completed cleanly: 739 of 739 documents succeeded, zero errors, in approximately 14.7 minutes (880 seconds). Full results are presented in Section 8 and Appendix C.

5.7 EVALUATION METRICS: FORMAL DEFINITIONS

For clarity and reproducibility, this subsection states, precisely, every metric reported in this paper.

Character Error Rate (CER) and Word Error Rate (WER)

CER and WER are both computed as the Levenshtein (minimum-edit-distance) distance between a hypothesis and a reference, normalized by the length of the reference, operating over characters for CER and over whitespace-delimited words for WER:

$$\text{CER} = (S + D + I) / N \quad \text{WER} = (S + D + I) / N$$

where S, D, I are the number of substitutions, deletions, and insertions required to transform the hypothesis into the reference, and N is the number of characters (CER) or words (WER) in the reference.

This paper reports corpus-level CER and WER: substitutions, deletions, and insertions are pooled across every document in the evaluation set before dividing, rather than computing a per-document CER/WER and averaging those values, because a per-document average would let a single very short (and disproportionately volatile) document skew the reported figure as heavily as a long one.

Top-1 Reconstruction Accuracy

The fraction of damaged words (`damage_status != intact`) whose `reconstructed_text` is an exact match to the ground-truth word, after stripping common trailing punctuation. Content-word accuracy restricts this computation to words that are not in a fixed function-word list (articles, prepositions, conjunctions, and common auxiliary verbs).

Expected Calibration Error (ECE)

Predictions are grouped into ten equal-width confidence bins. Within each bin, the absolute difference between the bin's average stated confidence and its observed (empirical) accuracy is computed, and ECE is the weighted average of that difference across bins, weighted by the number of predictions in each bin:

$$\text{ECE} = \sum_b (n_b / N) \cdot |\text{acc}(b) - \text{conf}(b)|$$

where b ranges over confidence bins, n_b is the number of predictions in bin b, N is the total number of predictions, $\text{acc}(b)$ is the observed accuracy in bin b, and $\text{conf}(b)$ is the average stated confidence in bin b.

Precision, Recall, and Unflagged Error Rate (Uncertainty Flagging)

Treating the uncertain flag as a binary classifier for “this reconstruction is wrong”: error recall is the share of actually-wrong reconstructions that were flagged uncertain; precision is the share of flagged reconstructions that were actually wrong; and unflagged error rate is the share of not-flagged reconstructions that were nonetheless wrong — the metric most directly relevant to a reviewer who only re-checks flagged words and otherwise trusts the rest.

5.8 TRAINING CONFIGURATION SUMMARY

For reproducibility, this subsection consolidates the fine-tuning configuration described piecemeal in Section 5.5.1 into a single reference point. Fine-tuning used the Hugging Face transformers Trainer API against the pretrained roberta-base checkpoint, with `transformers==4.44.2` and `tokenizers==0.19.1` pinned specifically to resolve the label-alignment corruption diagnosed in Section 5.5.1. Training ran on a single Kaggle-provided GPU instance, using the Trainer's default AdamW optimizer and linear learning-rate decay schedule, with `load_best_model_at_end=True` and validation loss as the tracked metric so that the checkpoint retained at the end of training was the best-observed one rather than simply the final epoch's weights. Every training run's configuration, including the corpus file hash it was trained against, is written to a run manifest at training time,

following the same verify-before-trusting principle (Section 4.5) applied everywhere else in this project — a training run's reported validation loss is only meaningful if the exact data it was computed against is also recorded.

Why roberta-base Rather Than a Larger Variant

roberta-base was chosen over larger variants (roberta-large or beyond) primarily for iteration speed: this project's development cycle depended on being able to fine-tune, evaluate, diagnose a bug, and re-fine-tune within a single working session on free-tier GPU compute, and a larger model would have made that cycle materially slower without changing the project's core question, which is about the reconstruction-and-confidence architecture rather than about pushing raw accuracy to its ceiling. Section 11 identifies scaling to a larger backbone as a concrete, low-risk direction for future work precisely because the architecture itself is not tied to model size.

6 SYSTEM ARCHITECTURE AND THE INTEGRATION SCHEMA

The pipeline's four functional stages (Section 4) communicate through one formally specified data contract: a JSON Schema (`document_schema.json`, draft 2020-12) that every document the pipeline produces is validated against before being considered complete. The schema's top-level object requires `document_id`, `source_image_path`, `page_number`, `words`, `confidence_threshold`, and `final_text`. Each entry in `words` describes exactly one word position and is required to carry `word_index`, `bounding_box`, `ocr_text`, `ocr_confidence`, `damage_status`, `damage_fraction`, `final_text`, and `provenance`.

Two conditional rules, expressed using JSON Schema's if/then construct, encode the core logic of the pipeline as machine-checkable constraints rather than only as prose:

- If a word's `damage_status` is "intact", then its `reconstructed_text`, `reconstruction_confidence`, and uncertain fields must be null, and its `provenance` must be "ocr".
- If a word's `damage_status` is "partial" or "missing", then its `provenance` must be "reconstructed", and `reconstructed_text`, `reconstruction_confidence`, and uncertain are all required (non-null) fields.

An optional evaluation object, present only in benchmark mode where ground truth is available, carries `ground_truth_text`, `cer`, `wer`, `reconstruction_top1_accuracy`, and `content_word_accuracy`. The last of these is declared with a `oneOf`: `[null, number]` type rather than a plain number, specifically to allow a document whose one targeted damaged word happens to be a function word (and therefore has no

content word to score) to legitimately report null rather than being forced into an invalid or misleading placeholder value — a fix made necessary by a real validation failure encountered during the first smoke test (Section 7.4).

The schema's own description field states its scope limitation explicitly: `damage_status` and `damage_fraction` are computed against a synthetically generated mask that the pipeline itself created, and are not a general damage-detection capability for genuinely unknown, real-world scanned documents. This is discussed further in Section 10. The full schema is reproduced in Appendix A.

6.1 SCHEMA VERSIONING AND EXTENSION

The schema is treated as a versioned contract rather than a fixed artifact. Its current draft (reproduced in Appendix A) intentionally omits an explicit schema-version field, which Section 11's proposed extensions (a genuine damage detector, joint reconstruction of adjacent damaged words) would need to introduce before they could safely coexist with documents produced under the current schema — for example, a future damage detector's own confidence score for a `damage_status` classification would need a new, optional field rather than overloading `damage_fraction`, which is currently defined purely as mask-overlap fraction and carries no notion of detector confidence. Because every pipeline output is validated against the schema at production time (Section 5.6.1), any such extension can be introduced additively — as new optional fields with sensible defaults — without invalidating documents already produced under the current schema, provided the validation step itself is updated in lockstep with the schema file. This additive-extension discipline is stated here explicitly so that future work building on this pipeline treats

the schema as a contract to extend carefully rather than a format to redesign from scratch.

6.2 WHY VALIDATION HAPPENS INSIDE THE PIPELINE, NOT AFTER

An alternative design would generate pipeline output first and validate it as a separate, later step — for example, as part of a batch quality-assurance pass. This project deliberately validates every document immediately after it is constructed, inside `build_document_json.py` itself (Section 5.6.2), rather than as an afterthought, for a reason directly illustrated by Section 7.4's schema/code type mismatch: that bug was caught on the very first smoke-test document specifically because validation ran inline and raised immediately, rather than allowing 20 (or 739) malformed documents to be written to disk first and discovered only during a later, separate audit. In a multi-stage pipeline, the cost of finding a data-contract violation grows with how far downstream of its origin it is discovered; validating inline keeps that cost at its minimum.

7 IMPLEMENTATION DETAILS AND ENGINEERING CHALLENGES

This section documents four specific engineering episodes from the project, each chosen because it illustrates a broader methodological point about building and validating a multi-stage machine learning pipeline rather than only reporting a final number.

7.1 THE MULTI-WORD RECONSTRUCTION BUG

Described fully in Section 5.6.3, this bug demonstrates that a component which is entirely correct in isolation (RoBERTa correctly assigning the highest joint probability to a fluent three-token sequence) can still produce an invalid output at the system level, because the task — reconstruct exactly one missing word — carries a constraint (no internal

word boundary) that the underlying model has no way of enforcing on its own. The lesson generalizes beyond this project: any pipeline that decodes an open-ended sequence from a subword-level language model and then treats that sequence as a single structured unit must explicitly validate the structural assumption, rather than trusting the model's own confidence ranking to respect it.

7.2 A COSMETIC ANOMALY, DIAGNOSED RATHER THAN ASSUMED

During evaluation, two otherwise identical pipeline runs produced final stitched text with a missing space in different locations — first between “of” and “reflection.”, then between “dispense” and “with” and “the.” Rather than accepting the first run's apparent reproducibility as proof the anomaly was a genuine, deterministic bug, `repr()` diagnostics were added to print the exact in-memory string representation of the two words at each junction. Both individual word strings printed cleanly as separate, correctly delimited strings ('of' and 'reflection.') on every run. Since the code path that joins words is deterministic and the individual strings were confirmed clean, but the location of the missing space moved between runs, the anomaly was concluded to be an artifact of how a terminal wraps and the user's own copy-paste of long JSON output — not a defect in the pipeline's actual in-memory string handling, which is what the CER/WER computation actually operates on. This conclusion was reached through direct evidence (the `repr()` output), not through the explicitly insufficient observation that a bug reproduced once.

7.3 RECONCILING A TEST-SET DENOMINATOR DISCREPANCY

At one point during development, a stated figure of 757 held-out test sentences was in circulation alongside newer figures of 739 and 1,478 test-set sizes drawn from different evaluation scripts. Rather

than silently adopting whichever number seemed most recent, `check_corpus.py` was written specifically to compute the SHA-256 hash of the corpus file on disk and count test-split records directly, at the moment of asking the question. This confirmed 739 as the correct, current test-set size — the figure used consistently in Step 5 — and additionally clarified that Step 4's 1,478-example figure and Step 5's 739-document figure are not the same measurement and are not intended to be reconciled to a single number: Step 4 evaluates 1,478 individual masked text examples (two per sentence, no image involved), while Step 5 evaluates 739 documents, each an image with exactly one deliberately damaged word. `run_held_out_evaluation.py`'s own manifest-writing logic now makes this check permanent: every full evaluation run records the corpus file's hash and the exact record count read, in `held_out_manifest.json`, so that the denominator behind any reported result is never assumed.

7.4 A SCHEMA/CODE TYPE MISMATCH, CAUGHT BY VALIDATION

The `content_word_accuracy` field was initially declared in the schema as a required, non-nullable number, but the batch evaluation script legitimately needs to emit null for any document whose one targeted damaged word turns out to be a function word (e.g., “the,” “of”), since there is then no content word to score. This mismatch was not caught by inspection; it was caught because the pipeline validates every output against the schema and the very first smoke-test document raised `jsonschema.exceptions.ValidationError: None is not of type 'number'`. The fix — changing the schema's declared type to `oneOf: [{"type": "null"}, {"type": "number", ...}]` and correspondingly fixing the evaluation script to compute a real 0.0/1.0/null value rather than a hardcoded placeholder — was verified by confirming that both the original

example instance and a new, deliberately null-content-word-accuracy instance validate successfully.

7.5 TOOLS, LIBRARIES, AND DEVELOPMENT ENVIRONMENT

The pipeline was developed and trained using the following tools and libraries:

- Python 3, as the implementation language for every pipeline component.
- Tesseract OCR (via the `pytesseract` wrapper) for optical character recognition.
- PyTorch and the Hugging Face transformers library (with tokenizers pinned to a compatible version, see Section 5.5.1) for loading, fine-tuning, and running the RoBERTa reconstruction model.
- `jiwer` for corpus-level CER/WER computation.
- `jsonschema` for validating every pipeline output against the JSON Schema integration contract (Section 6, Appendix A).
- Pillow (PIL) for image loading, synthetic damage rendering, and background-color estimation.
- Kaggle's free GPU-backed notebook environment for model fine-tuning, with the fine-tuned model weights subsequently downloaded to a local machine for Step 5 integration and evaluation.
- Git and GitHub for version control and final project publication, with the fine-tuned model weights (too large for GitHub's file-size limit) hosted separately on Google Drive and linked from the repository README.

This project's own documentation practice is itself part of its methodology: every development stage produced a dedicated results note (title block, identity table, numbered sections, matching visual style across all five), each independently verified by

rendering it to PDF and visually inspecting every page before it was considered complete — the same verification discipline applied to this paper itself.

7.6 ERROR HANDLING AND FAILURE MODES

Beyond the specific bugs documented above, the stitching pipeline (`build_document_json.py`) was written with explicit handling for three categories of failure that a batch process run against hundreds of documents will eventually encounter even when every individual component is correct in isolation.

Missing or Unmatched OCR Detections

Because word matching (Section 5.6.2) relies on Intersection-over-Union overlap between an OCR-detected bounding box and a known expected word position, a word whose damage is severe enough that Tesseract detects no bounding box at all in that region has no OCR candidate to match against. The pipeline treats this explicitly as a zero-overlap match rather than allowing it to raise an unhandled exception, which routes the word into the same reconstruction path as a lower-confidence partial match, since from the pipeline's point of view a wholly undetected word and a severely mismatched one both mean “trust the OCR text for this position” is not a safe default.

Model Load Fallback

As noted in Section 5.6.2, if the fine-tuned model weights are not found at the expected local path, the pipeline falls back to the pretrained roberta-base checkpoint rather than failing outright, but does so with an explicit, loud warning written to the run log, because the locked 0.5442 confidence threshold (Section 5.5.3) was calibrated specifically against the fine-tuned model's confidence distribution and is not guaranteed to carry the same operational meaning for the pretrained model's confidence distribution. This fallback exists to keep the pipeline runnable in a fresh environment (for example, this

repository cloned without first downloading the separately hosted model weights, Section 7.5) without ever silently reporting flag-rate or precision figures that were quietly computed against the wrong model.

Per-Document Isolation During Batch Evaluation

`run_held_out_evaluation.py` wraps the processing of each of the 739 test documents in its own error boundary and writes each document's result to disk immediately (Section 5.6.4), rather than accumulating results in memory and writing them only at the end of the run. This means a single malformed or unexpectedly structured document cannot silently corrupt or halt the evaluation of the remaining 738, and a partial run interrupted partway through leaves a complete, valid, already-flushed set of results for every document processed so far. In the actual full run reported in this paper, this isolation was not needed to recover from any failure — all 739 documents succeeded — but it was treated as a required safety property of the evaluation script regardless of whether that run happened to need it.

7.7 TESTING PRACTICE ACROSS THE PROJECT

Two levels of testing were used throughout development, distinct from the held-out evaluation described in Section 5.6.4, which measures accuracy rather than correctness. First, targeted unit-style checks validated individual functions in isolation against hand-constructed cases with a known correct answer — for example, confirming that `classify_all_words` (Section 5.3) correctly labels a word as intact, partial, or missing at overlap fractions chosen specifically to sit just inside and just outside each threshold boundary, and confirming that `match_ocr_word` (Appendix B.2) returns `None` rather than a spurious match when no candidate box clears the minimum IoU overlap. Second, schema-level tests validated the JSON Schema itself

(Section 5.6.1) against both a fully populated, valid example instance and deliberately broken instances constructed to violate specific if/then rules, confirming that the schema actually rejects the violations it claims to enforce rather than merely documenting them in prose. Neither level of testing substitutes for the full 739-document held-out evaluation, which remains the paper's primary evidence of system-level correctness and accuracy; unit- and schema-level tests instead reduce the search space when a full-pipeline run does fail, by ruling out entire categories of bugs (a boundary condition, a schema rule that does not actually fire) before debugging turns to the harder question of how components interact.

8 RESULTS

8.1 STEP 4 — TEXT-ONLY CALIBRATION AND TEST-SET TRANSFER

Metric	Validation set	Test set (1,478 examples)
Error recall (share of wrong reconstructions flagged)	80.06%	77.21%
Precision (share of flagged reconstructions that are wrong)	79.61%	78.98%
Flag rate	53.45%	50.20%
Unflagged error rate	—	~23.5%
Unfiltered baseline error rate	—	51.35%

Expected Calibration Error (10 bins): 0.0704. The model is overconfident in every bin, including the highest-confidence bin (96.9% average confidence vs. 94.9% observed accuracy).

Paired bootstrap (10,000 resamples) on the Top-1 accuracy improvement from fine-tuning plus retraining with `load_best_model_at_end=True`: +0.47 percentage points, 95% CI [-3.04, +4.06] — not statistically detectable at this sample size, though compatible with a real effect of up to roughly four points.

Retraining outcome: validation MLM loss fell from 2.2367 to 1.6615; whole-word reconstruction accuracy improved from 48.17%/26.90% to 48.65%/27.03% (overall / content-word).

8.2 STEP 5 — FULL HELD-OUT EVALUATION (N = 739 DOCUMENTS)

Metric	Value
Documents attempted	739
Documents succeeded	739 (100%)
Documents errored	0
Total run time	880 seconds (~14.7 minutes)
OCR-only corpus-level CER	4.92%
Final stitched corpus-level CER	3.00%
OCR-only corpus-level WER	5.60%
Final stitched corpus-level WER	3.83%
Reconstructed words (n)	781
Reconstruction Top-1 accuracy (overall)	40.32%
Content-word Top-1 accuracy (n = 521)	28.02%
Uncertainty flag rate (locked threshold 0.5442)	46.55%
Error recall (locked threshold)	65.76%
Precision (locked threshold)	84.30%
Unflagged error rate (locked threshold)	38.23%

Metric	Value
Total words in corpus	15,680
Words with OCR provenance	14,899
Words with reconstructed provenance	781

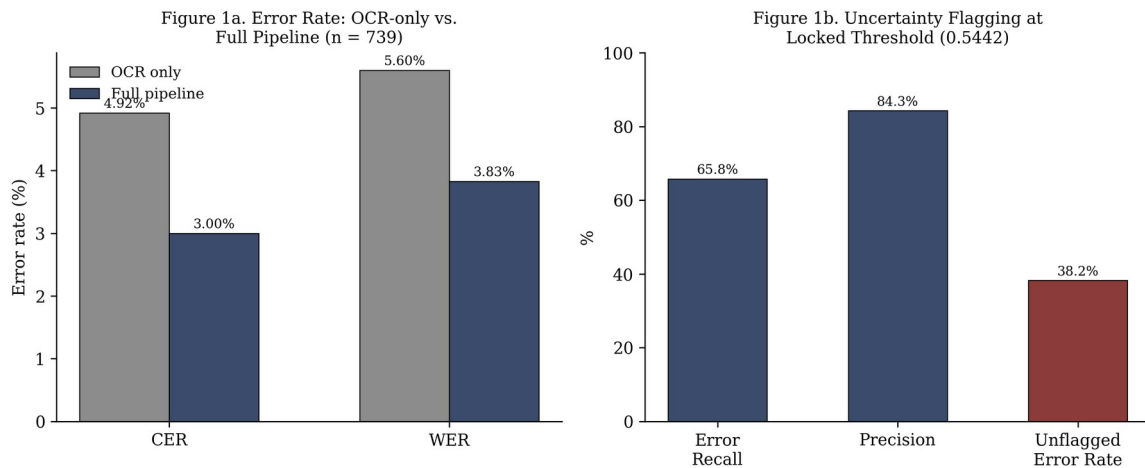


Figure 1. Left: corpus-level CER and WER, OCR-only vs. the full pipeline, across all 739 held-out documents. Right: error recall, precision, and unflagged error rate of the uncertainty-flagging layer at its locked confidence threshold (0.5442), evaluated under real image-based conditions.

These results show the full pipeline meaningfully reduces both CER and WER relative to OCR alone, and that the confidence layer, while its recall drops relative to its Step 4 text-only calibration, still substantially concentrates true errors among flagged words (84.30% precision) rather than flagging indiscriminately.

8.3 SUMMARY COMPARISON ACROSS DEVELOPMENT STAGES

Stage	What was measured	Headline result
Step 1	OCR accuracy under synthetic degradation	Confirms OCR degrades sharply and silently under damage
Step 3	Pretrained reconstruction (no fine-tuning)	Fluent but frequently wrong; establishes lower bound
Step 4	Fine-tuned reconstruction + calibrated confidence	+0.47-pt accuracy gain (not statistically detectable); ECE 0.0704; 77.21% recall on text-only test set
Step 5	Full end-to-end pipeline, image-based, n=739	CER 4.92%→3.00%, WER 5.60%→3.83%; 65.76% recall / 84.30% precision

8.4 QUALITATIVE OBSERVATIONS

Beyond the aggregate figures in Sections 8.1 and 8.2, manually inspecting individual reconstructions surfaced two recurring qualitative patterns worth recording alongside the numbers. First, the pretrained-versus-fine-tuned contrast described in Section 5.4 is visible at the level of individual examples, not only in aggregate accuracy: the pretrained baseline consistently produces a locally fluent word (e.g., “convenient”) that reads naturally in context but is not the specific word originally present, whereas the fine-tuned model's errors,

when it errs, more often fall on function words and short common words where several completions are genuinely plausible from context alone — a harder class of error to eliminate through fine-tuning alone, since the surrounding text sometimes underdetermines the correct word even for a careful human reader. Second, the single-word validity fix described in Section 5.6.3 changed not just whether a reconstruction was structurally valid, but also its reported confidence: the invalid three-token candidate (“choices, and”) that spanned a word boundary carried a confidence of 0.193, while the corrected, structurally valid candidate (“choices;”) for the same position carried a lower confidence of 0.1069, illustrating that a structurally invalid candidate can appear more confident than a valid one purely because it was never constrained to respect word boundaries in the first place — a concrete demonstration of why the validity check in Section 5.6.3 needed to run before, not after, candidates are compared by confidence.

A third observation concerns the relationship between damage severity and reconstruction difficulty: words classified missing (more than 90% mask overlap, Section 5.3) offer the reconstruction model no partial visual or OCR evidence at all, and rely entirely on the surrounding text's linguistic context, whereas words classified partial retain some OCR signal that, while unreliable, is not entirely absent. This project's evaluation does not separately break out reconstruction accuracy by damage severity tier, and doing so is noted in Section 11 as a natural extension that would clarify whether the aggregate 40.32% Top-1 accuracy figure masks a meaningful difference between these two damage categories.

9 DISCUSSION

9.1 THE GENERALIZATION GAP

The single most important analytical finding of this project is not a headline accuracy number but a generalization gap: the confidence threshold locked during Step 4, using purely text-only masked examples, does not transfer fully to the real image-based, OCR-noise-affected conditions of Step 5. Error recall falls from 77.21% (Step 4 test set) to 65.76% (Step 5), while precision actually improves slightly (78.98% to 84.30%), and the unflagged error rate nearly doubles, from roughly 23.5% to 38.23%. This is reported here as a substantive, reportable result, not as a discrepancy to be explained away or hidden: it demonstrates concretely that a confidence layer calibrated under one data distribution (clean text, synthetically masked) cannot be assumed reliable under a materially different distribution (real OCR output on damaged images, with its own noise characteristics) without being re-validated there. This is precisely the kind of finding that only an end-to-end, image-based evaluation — as opposed to stopping at Step 4's text-only benchmark — can surface, and it is presented as one of this paper's central contributions.

The practical implication for a deployment scenario is that the current threshold (0.5442) still provides real, positive value — an archivist relying on it filters out a large share of true errors at high precision, and unflagged output is meaningfully cleaner than unfiltered output — but the specific recall/precision trade-off should not be assumed to hold exactly once the system is deployed on a different, real-world damage distribution than the one it was benchmarked on here. Section 11 proposes recalibrating the threshold directly on real-image evaluation data as the most direct fix.

9.2 RECONSTRUCTION ACCURACY IN CONTEXT

A second discussion point concerns reconstruction accuracy in absolute terms: 40.32% Top-1 accuracy (28.02% on content words specifically) means the majority of individual reconstructed words are not exact matches to ground truth. This should be read alongside the corpus-level CER/WER improvement, which is still substantial (CER nearly halved, from 4.92% to 3.00%) because reconstruction only ever touches the small fraction of words the pipeline has classified as damaged, and because even an imperfect reconstruction is frequently closer, at the character level, to the true word than the raw, damage-corrupted OCR output it replaces.

9.3 ETHICAL AND PRACTICAL CONSIDERATIONS FOR ARCHIVAL USE

A system that reconstructs missing text and presents it alongside genuinely observed OCR output carries a specific risk that is worth stating plainly: a reconstructed word is a statistically informed guess, not a recovered fact, and nothing in the pipeline's output should be mistaken for the latter. Three design choices in this project follow directly from that concern. First, every reconstructed word's provenance is recorded explicitly (provenance: "reconstructed") in the schema, so that a reconstructed word can never be silently indistinguishable, in the output record, from a word the OCR engine actually observed on the page. Second, the uncertain flag exists specifically so that a human reviewer — an archivist, a records officer, a researcher — is pointed toward the reconstructions least likely to be trustworthy, rather than being asked to review every word uniformly or none at all. Third, and most importantly, this paper documents the confidence layer's real limitations (Section 9.1, Section 10) rather than presenting its validation-set performance as a universal guarantee, precisely

because overstating a system's reliability in an archival or legal-record context can cause real harm: a fabricated word accepted as genuine, in a court record, a land title, or a historical archive, is a more serious failure than the same system simply declining to reconstruct a word it is unsure about. Any deployment of this pipeline should preserve the original damaged-image scan alongside any reconstructed transcription, and should present reconstructed text as a labeled aid to a human reader rather than as a substitute for verifying the original document.

9.4 THREATS TO VALIDITY

Internal Validity

The corpus split (train/validation/test) is performed at the sentence level, which is sufficient to prevent the exact sentences used for fine-tuning from reappearing in evaluation, but does not guarantee that no lexical or stylistic overlap exists between splits, since all three are drawn from the same small set of public-domain source texts. A held-out test set drawn from entirely different authors or genres would be a stronger internal-validity check.

External Validity

Every result in this paper is conditioned on one damage model (a background-color-matched ribbon tear), one OCR engine (Tesseract), one reconstruction backbone (RoBERTa-base), and one language (English). None of the headline figures in Section 8 should be assumed to generalize to a different damage type, OCR engine, model family, or language without separate evaluation — this is the same caution already stated for the confidence threshold specifically (Section 9.1) but applies to the whole pipeline's accuracy figures as well.

Construct Validity

Top-1 exact-match accuracy (Section 5.7) is a strict measure: a reconstruction that is semantically

reasonable but not an exact string match to ground truth is scored as wrong, even where a human reader might judge it an acceptable recovery. The corpus-level CER/WER figures partially compensate for this, since a close-but-not-exact reconstruction is still rewarded at the character level, but no metric in this paper directly measures semantic acceptability as a human archivist would judge it.

9.5 COMPARISON TO A NAIVE BASELINE

It is worth stating explicitly what the reported CER/WER improvement is, and is not, being compared against. The 4.92%-to-3.00% CER reduction is measured against raw, unmodified OCR output on the same damaged images — not against a stronger classical baseline such as spell-checking or dictionary-based correction of the OCR output. A dictionary-based corrector would plausibly recover some of the same errors this pipeline recovers (particularly where the damaged word is a common word with few plausible completions) at much lower computational cost, and would not require a fine-tuned language model at all. This project did not implement or evaluate that baseline, and Section 11 lists it as a natural, low-effort addition: running a simple dictionary/edit-distance correction step alongside the language-model reconstruction, and reporting whether the language model's added complexity earns a measurable improvement over the cheaper alternative on this specific damage distribution, would meaningfully strengthen the case for the more expensive architecture used here.

9.6 WHAT A PRACTITIONER SHOULD TAKE AWAY

For a reader deciding whether an architecture like this one is appropriate for their own archival digitization project, three practical conclusions follow directly from the results in Section 8. First, the corpus-level error-rate improvement (CER nearly halved) is real and measured end-to-end on images, not only on an isolated text benchmark, which is the

strongest form of evidence this paper can offer for the pipeline's core value proposition. Second, the confidence layer's threshold and its recall/precision behavior should be treated as specific to the damage distribution and OCR engine it was calibrated and evaluated on, and should be re-validated, not merely reused, against a different institution's own scan quality and damage profile before being trusted operationally (Section 9.1). Third, the system's explicit provenance tagging and human-review flagging are not optional conveniences but the mechanism that makes the whole approach safe to use in an archival context at all (Section 9.3); a deployment that strips out the confidence layer and treats every reconstruction as equally trustworthy would be adopting this project's accuracy numbers without its safety guarantees.

9.7 RELATIONSHIP TO HUMAN-IN-THE-LOOP SYSTEMS

The design described in this paper is, by construction, a human-in-the-loop system rather than a fully autonomous one: the uncertain flag exists specifically to route a defined subset of reconstructions to a human reviewer rather than to make every decision unilaterally. This has a direct, quantifiable implication for how the system should be evaluated as part of a real workflow: the relevant question for an adopting institution is not only “what is the pipeline's raw accuracy,” but “given a reviewer with a fixed amount of time, how much of the archive's true error can that reviewer catch by checking only the flagged words.” Section 8.2's 65.76% error recall at a 46.55% flag rate is exactly this trade-off stated in those terms: a reviewer who checks slightly under half of all reconstructed words catches roughly two-thirds of the actual errors among them. Whether that trade-off is acceptable depends on factors this paper does not attempt to model — the cost of an uncaught error in the reviewing institution's specific context, the

reviewer's available time, and the volume of documents being processed — and is deliberately left as an operational decision for the adopting institution rather than a judgment this paper makes on its behalf. What this paper does commit to is making that trade-off visible and quantified, rather than hidden inside an opaque accuracy number, which is the same underlying principle stated as a design goal in Section 4.5 and Section 9.3.

10 LIMITATIONS AND SCOPE

This paper documents its system's limitations explicitly, following the project's stated design principle that “the final stage will not invent fields that no component can support yet.”

10.1 Damage Location Is Known, Not Detected

The damage mask used throughout this project is generated synthetically by the pipeline itself for benchmarking purposes, so `damage_status` and `damage_fraction` are computed against a known ground truth. No component in this pipeline detects damage location in a genuinely unknown, real-world scanned document with no reference mask. This is the single largest gap between the current system and real deployment readiness.

10.2 Reconstruction Is One Word at a Time

Adjacent damaged words are reconstructed independently; a damaged word's immediate neighbor, if also damaged, contributes only a placeholder token as context rather than its own (not-yet-computed) reconstruction. Because every test document in this evaluation contains exactly one damaged word, this code path has not been exercised or evaluated.

10.3 Target Subword Length Is Searched, Not Known

In real use there is no ground-truth word to tokenize in advance, so reconstruction searches over one,

two, and three mask tokens and keeps the highest-confidence result, rather than being told the correct length in advance, as Step 4's isolated benchmark evaluation was.

10.4 The Confidence Threshold Is Calibrated on Text, Evaluated on Images

As discussed in Section 9, the locked threshold's recall and precision shift meaningfully between the text-only calibration setting and the real image-based evaluation setting.

10.5 Single-Language, Single-Domain Corpus

The corpus is drawn from English-language, public-domain literary text. Performance on other languages, domains (e.g., handwriting, technical documents, forms), or degradation types (e.g., water damage, mould, ink bleed rather than clean tears) has not been evaluated.

10.6 No Comparison Against a Non-Neural Baseline

As discussed in Section 9.5, this project measures its improvement against raw OCR output rather than against a cheaper dictionary- or edit-distance-based correction baseline. The reported gains should therefore be read as “better than doing nothing,” not yet as “better than the cheapest reasonable alternative,” pending the comparison proposed in Section 11.

10.7 Single-Run Fine-Tuning

The fine-tuning run reported in Section 5.5.1 was not repeated across multiple random seeds. The reported validation loss and reconstruction-accuracy figures therefore reflect one training run's outcome rather than a mean and variance across repeated runs, and the bootstrap confidence interval reported in Section 5.5.3 quantifies uncertainty in the evaluation-set measurement, not uncertainty arising from training-run-to-training-run

variance, which is a distinct and unmeasured source of variability.

10.8 Compute Environment Dependency

Fine-tuning was performed on Kaggle's free, shared GPU-backed notebook environment (Section 5.8), a convenient but non-dedicated compute resource whose exact hardware allocation and availability are not guaranteed to be identical across sessions. Exactly reproducing this paper's reported training loss and timing figures on a different GPU model, driver version, or PyTorch build is not guaranteed, even with the pinned library versions recorded in Appendix D; what is preserved across environments is the training procedure and the corpus itself, not necessarily bit-for-bit identical numerical output, which is a common and generally accepted limitation of deep-learning reproducibility rather than one specific to this project.

11 FUTURE WORK

- Recalibrate the confidence threshold on real-image evaluation data. Rather than relying on a threshold locked from text-only calibration, repeat the Step 4 calibration methodology (binning, ECE, validation-only threshold selection) directly on a validation split of image-based, OCR-derived reconstructions, so that the locked threshold's recall/precision guarantees hold under the conditions it will actually be deployed in.
- Combine OCR confidence and reconstruction confidence into a single signal. Currently, uncertain is computed purely from reconstruction confidence for damaged words, and is null for intact words. A unified confidence signal that also incorporates Tesseract's own word-level OCR confidence could catch cases where OCR itself is unreliable even on words the damage mask classifies as intact.

- Build a genuine damage-region detector. Replacing the synthetic-mask-dependent damage classification with a learned detector (for example, a segmentation model trained on examples of real torn, stained, or faded documents) is the most direct way to close the largest scope gap identified in Section 10.
- Joint reconstruction of adjacent damaged words. Extend the reconstruction step to condition on other damaged words' own (in-progress) reconstructions rather than a placeholder, and evaluate this specifically on documents with multiple adjacent damaged words, which the current 739-document evaluation does not include by design.
- Extend beyond clean synthetic tears. Evaluate and, if necessary, retrain the pipeline against a wider range of degradation types — water and mould staining, ink bleed-through, and genuine historical wear — rather than the ribbon-tear synthetic damage used throughout this project.
- Benchmark against a non-neural correction baseline. Implement a simple dictionary- or edit-distance-based word correction step and report its accuracy and cost alongside the language-model reconstruction, directly addressing the comparison gap identified in Section 9.5 and Section 10.6.
- Repeat fine-tuning across multiple seeds. Re-run Step 4's fine-tuning procedure with several different random seeds and report the mean and spread of validation loss and reconstruction accuracy, to separate genuine architectural improvement from ordinary training-run variance (Section 10.7).
- Scale the reconstruction backbone. Evaluate whether a larger pretrained model (roberta-large or a comparably sized alternative) meaningfully improves reconstruction accuracy over roberta-base, now that the architecture and evaluation

pipeline built in this project make that comparison a matter of re-running Steps 4 and 5 with a different checkpoint rather than new infrastructure work (Section 5.8).

- Build a review interface. Design and implement a minimal reviewer-facing interface that displays a page image alongside its JSON output, visually highlighting flagged (uncertain) words, so that the confidence layer's practical value (Section 9.6) can be evaluated with an actual human reviewer rather than only through offline precision/recall metrics.
- Break out accuracy by damage severity tier. Report reconstruction accuracy and confidence-flagging performance separately for partially damaged versus fully missing words (Section 8.4), to establish whether the aggregate accuracy figure masks a meaningful difference between these two damage categories.

12 CONCLUSION

This paper has documented the design, implementation, and evaluation of a five-stage pipeline for recovering text from damaged or poorly scanned documents, built incrementally from a baseline OCR characterization through to a fully integrated, schema-validated, end-to-end system evaluated on 739 held-out documents. The completed pipeline reduces character error rate from 4.92% to 3.00% and word error rate from 5.60% to 3.83% relative to OCR alone, while its calibrated uncertainty layer flags 65.76% of reconstruction errors at 84.30% precision under real, image-based evaluation conditions.

Just as importantly, the project surfaced and documented, rather than hid, a genuine generalization gap between the confidence layer's text-only calibration and its real-world transfer performance, together with every bug found during integration testing and the exact diagnostic

evidence used to resolve each one. The system's remaining limitations — chiefly, that damage location is currently known rather than detected — are stated explicitly, together with a concrete plan for addressing them, so that this paper's contribution is not only a working pipeline but a fully reproducible, honestly scoped account of what that pipeline does and does not yet do.

12.1 SUMMARY OF CONTRIBUTIONS

- A working, end-to-end, schema-validated pipeline, evaluated on all 739 documents in its held-out test set with zero runtime errors (Section 8.2).
- A measured, corpus-level reduction in both character and word error rate relative to raw OCR, verified end-to-end on real images rather than only on an isolated text benchmark (Section 8.2).
- A calibrated confidence layer whose real, image-based transfer performance is reported honestly against its text-only calibration baseline, surfacing a generalization gap treated as a primary finding of this paper rather than a discrepancy to reconcile away (Section 9.1).
- A fully reproducible evaluation methodology, in which every headline figure is traceable to a specific script, a corpus file hash, and a record count captured at the moment of the run (Section 7.3, Appendix D, Appendix E).
- A candid account of the system's scope limitations and a concrete, prioritized plan for closing them, so that a reader can adopt, extend, or critique this work from a position of full information about what it does and does not yet do (Sections 10 and 11).

13 REFERENCES

- evidence used to resolve each one. The system's remaining limitations — chiefly, that damage location is currently known rather than detected — are stated explicitly, together with a concrete plan for addressing them, so that this paper's contribution is not only a working pipeline but a fully reproducible, honestly scoped account of what that pipeline does and does not yet do.
- International Conference on Document Analysis and Recognition (ICDAR 2007), Vol. 2, pp. 629–633. IEEE.
- [2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv:1810.04805.
- [3] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv preprint arXiv:1907.11692.
- [4] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. In Proceedings of the 34th International Conference on Machine Learning (ICML 2017). arXiv preprint arXiv:1706.04599.
- [5] OCR Error Post-Correction with LLMs in Historical Documents: No Free Lunches. (2025). Proceedings of the ACL Workshop on Resource-Constrained NLP (Resourceful 2025). arXiv preprint arXiv:2502.01205.
- [6] JSON Schema Draft 2020-12 Specification. IETF / json-schema.org.
- [7] Austen, J. *Pride and Prejudice* (public domain edition), used as a source text for corpus construction.
- [1] Smith, R. (2007). An Overview of the Tesseract OCR Engine. In Proceedings of the Ninth

APPENDIX A — FULL JSON INTEGRATION SCHEMA

The complete document_schema.json (JSON Schema draft 2020-12) used to validate every document produced by the pipeline, reproduced in full below.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://learndepth.local/ml-t2-022/document-schema.json",
  "title": "Reconstructed Document Output",
  "description": "The integration contract for Step 5. One instance of this
  schema is produced per processed page. Each pipeline component (OCR, damage
  classification, reconstruction, confidence scoring) fills in only the fields it
  is responsible for; the final stitching stage combines them and invents nothing
  new. NOTE ON SCOPE: 'damage_status' and 'damage_fraction' are currently derived
  by comparing each word's bounding box against a synthetically generated tear
  mask (Step 2), which is known because the pipeline created the damage itself for
  benchmarking. No component in this pipeline yet detects damage location in a
  real scanned document with no known mask; that is out of scope for this schema
  and remains future work. 'evaluation' is only present when this document was
  processed in benchmark mode, where ground truth text is available.",
  "type": "object",
  "required": ["document_id", "source_image_path", "page_number", "words",
  "confidence_threshold", "final_text"],
  "additionalProperties": false,
  "properties": {
    "document_id": {
      "type": "string",
      "description": "Stable identifier for the source document, assigned at
  ingestion."
    },
    "source_image_path": {
      "type": "string",
      "description": "Path to the source page image this output was generated
  from."
    },
    "page_number": {
      "type": "integer",
      "minimum": 1,
      "description": "1-indexed page number within the source document."
    },
    "confidence_threshold": {
      "type": "number",
      "minimum": 0,
      "maximum": 1,
      "description": "The locked reconstruction-confidence threshold used to
  compute each word's 'uncertain' flag (Step 4: chosen on a validation set to
  catch >=80% of wrong reconstructions, then locked). Included per-document so the
  document is self-describing even if the threshold is retuned later."
    },
    "final_text": {
      "type": "string",
      "description": "The full stitched output text for the page: each word's
  'final_text' joined in reading order."
    }
  }
}
```



```

    "words": {
      "type": "array",
      "description": "One entry per word position on the page, in reading
order.",
      "items": { "$ref": "#/$defs/word" }
    },
    "evaluation": {
      "oneOf": [
        { "type": "null" },
        { "$ref": "#/$defs/evaluation" }
      ],
      "description": "Present only when ground truth text is available for this
document (benchmark/test mode). Null or omitted in real deployment, where no
ground truth exists."
    }
  },
  "$defs": {
    "bounding_box": {
      "type": "object",
      "required": ["left", "top", "width", "height"],
      "additionalProperties": false,
      "properties": {
        "left": { "type": "integer", "minimum": 0 },
        "top": { "type": "integer", "minimum": 0 },
        "width": { "type": "integer", "minimum": 0 },
        "height": { "type": "integer", "minimum": 0 }
      },
      "description": "Pixel bounding box, same convention as
pytesseract.image_to_data: left/top of the box's top-left corner, plus width and
height."
    },
    "word": {
      "type": "object",
      "required": [
        "word_index", "bounding_box", "ocr_text", "ocr_confidence",
        "damage_status", "damage_fraction", "final_text", "provenance"
      ],
      "additionalProperties": false,
      "properties": {
        "word_index": {
          "type": "integer",
          "minimum": 0,
          "description": "0-indexed position of this word in reading order on
the page."
        },
        "bounding_box": { "$ref": "#/$defs/bounding_box" },
        "ocr_text": {
          "type": ["string", "null"],
          "description": "Raw text Tesseract returned at this position on the
(possibly damaged) image. Null when OCR detected no text here at all, which
happens when a word is torn away entirely (damage_status: missing)."
        },
        "ocr_confidence": {
          "type": ["number", "null"],
          "minimum": 0,

```

```

    "maximum": 100,
    "description": "Tesseract's own word-level confidence, native 0-100
scale, not renormalized. Null when ocr_text is null."
  },
  "damage_status": {
    "type": "string",
    "enum": ["intact", "partial", "missing"],
    "description": "From Step 2/3's mask-coverage classification: intact
(<5% of the word's box overlaps the damage mask), partial (5-90%), missing
(>90%). See top-level description for the scope limitation on how this is
currently computed."
  },
  "damage_fraction": {
    "type": "number",
    "minimum": 0,
    "maximum": 1,
    "description": "Fraction of the word's bounding box overlapping the
damage mask, pixel-by-pixel."
  },
  "reconstructed_text": {
    "type": ["string", "null"],
    "description": "The model's reconstructed word. Present only when
damage_status is 'partial' or 'missing' (reconstruction is attempted for any
non-intact word); null/absent for intact words, which keep their OCR text as-
is."
  },
  "reconstruction_confidence": {
    "type": ["number", "null"],
    "minimum": 0,
    "maximum": 1,
    "description": "Geometric mean of the per-subword probabilities
RoBERTa assigned its own chosen tokens while decoding this word (Step 4):
exp(mean(log p_i)). Present only when reconstructed_text is present."
  },
  "uncertain": {
    "type": ["boolean", "null"],
    "description": "True when reconstruction_confidence is at or below
confidence_threshold, i.e. flagged for human review. Present only when
reconstruction_confidence is present; null for intact words, since the Step 4
confidence layer was built specifically for reconstruction output, not raw OCR
confidence."
  },
  "final_text": {
    "type": "string",
    "description": "The text actually used for this word in the stitched
output: reconstructed_text if reconstruction ran, otherwise ocr_text."
  },
  "provenance": {
    "type": "string",
    "enum": ["ocr", "reconstructed"],
    "description": "Which source final_text came from."
  }
},
"allof": [
  {

```

```

    "if": { "properties": { "damage_status": { "const": "intact" } } },
    "then": {
      "properties": {
        "reconstructed_text": { "const": null },
        "reconstruction_confidence": { "const": null },
        "uncertain": { "const": null },
        "provenance": { "const": "ocr" }
      }
    },
  },
  {
    "if": { "properties": { "damage_status": { "enum": ["partial",
"missing"] } } },
    "then": {
      "properties": {
        "provenance": { "const": "reconstructed" }
      },
      "required": ["reconstructed_text", "reconstruction_confidence",
"uncertain"]
    }
  }
],
"evaluation": {
  "type": "object",
  "required": ["ground_truth_text", "cer", "wer",
"reconstruction_top1_accuracy", "content_word_accuracy"],
  "additionalProperties": false,
  "properties": {
    "ground_truth_text": {
      "type": "string",
      "description": "The known clean text this page was generated from,
used only in benchmark/test mode."
    },
    "cer": { "type": "number", "minimum": 0, "description": "Character error
rate of final_text against ground_truth_text." },
    "wer": { "type": "number", "minimum": 0, "description": "Word error rate
of final_text against ground_truth_text." },
    "reconstruction_top1_accuracy": {
      "type": "number", "minimum": 0, "maximum": 1,
      "description": "Share of words with damage_status != intact whose
reconstructed_text exactly matches ground truth."
    },
    "content_word_accuracy": {
      "oneOf": [
        { "type": "null" },
        { "type": "number", "minimum": 0, "maximum": 1 }
      ],
      "description": "Same as reconstruction_top1_accuracy, restricted to
content words. Null when this document's reconstructed word(s) happened to be
entirely function words, so there is nothing content-word-specific to score."
    }
  }
}
}
}

```

```
}
```

APPENDIX B — KEY SOURCE CODE LISTINGS

B.1 RECONSTRUCTION WITH SINGLE-WORD VALIDITY CONSTRAINT

From `build_document_json.py`. This is the corrected version, after the multi-word reconstruction bug described in Section 5.6.3 and Section 7.1 was fixed.

```
def reconstruct_word(tokenizer, model, context_words, target_index):
    """Iterative masked-token decoding for one word position."""
    import torch
    before_text = " ".join(context_words[:target_index])
    after_text = " ".join(context_words[target_index + 1:])

    best_result = None
    for num_mask_tokens in (1, 2, 3):
        predicted_ids, log_probs = [], []
        running_before = before_text
        for step in range(num_mask_tokens):
            remaining = num_mask_tokens - step
            mask_span = " ".join([tokenizer.mask_token] * remaining)
            full_text = f"{running_before} {mask_span} {after_text}".strip()
            encoding = tokenizer(full_text, truncation=True, max_length=64,
return_tensors="pt")
            input_ids = encoding["input_ids"][0]
            mask_positions = (input_ids ==
tokenizer.mask_token_id).nonzero(as_tuple=True)[0].tolist()
            if len(mask_positions) != remaining:
                break
            with torch.no_grad():
                outputs = model(**encoding)
                logits = outputs.logits[0]
                probs = torch.softmax(logits[mask_positions[0]], dim=-1)
                predicted_id = torch.argmax(probs).item()
                predicted_ids.append(predicted_id)
                log_probs.append(math.log(probs[predicted_id].item() + 1e-12))
                running_before = tokenizer.decode(
                    tokenizer(running_before, add_special_tokens=False)["input_ids"]
+ [predicted_id]
                )
            if len(predicted_ids) != num_mask_tokens:
                continue
            word = tokenizer.decode(predicted_ids).strip()
            if " " in word:
                continue # spans a word boundary - not a valid single-word
candidate
            confidence = math.exp(sum(log_probs) / len(log_probs))
            if best_result is None or confidence > best_result[1]:
                best_result = (word, confidence)

    if best_result is None:
```

```

        return "[reconstruction failed]", 0.0
    return best_result

```

B.2 IoU-BASED OCR WORD MATCHING

From `build_document_json.py`. Matches each expected ground-truth word position to the OCR-detected word (on the damaged image) with the highest bounding-box overlap.

```

def match_ocr_word(box, ocr_words, min_overlap=0.3):
    """Finds the OCR-detected word (on the damaged image) whose box
    overlaps 'box' the most, by intersection-over-union. Returns None
    when nothing overlaps enough - expected for words OCR could not
    read at all under heavy damage."""
    bx1, by1 = box["left"], box["top"]
    bx2, by2 = bx1 + box["width"], by1 + box["height"]
    b_area = box["width"] * box["height"]

    best, best_iou = None, 0.0
    for w in ocr_words:
        wx1, wy1 = w["left"], w["top"]
        wx2, wy2 = wx1 + w["width"], wy1 + w["height"]
        ix1, iy1 = max(bx1, wx1), max(by1, wy1)
        ix2, iy2 = min(bx2, wx2), min(by2, wy2)
        if ix2 <= ix1 or iy2 <= iy1:
            continue
        inter = (ix2 - ix1) * (iy2 - iy1)
        union = b_area + w["width"] * w["height"] - inter
        iou = inter / union if union > 0 else 0.0
        if iou > best_iou:
            best, best_iou = w, iou

    return best if best_iou >= min_overlap else None

```

APPENDIX C — FULL STEP 5 AGGREGATE RESULTS

The complete aggregate summary report (`held_out_summary_report.json`) produced by `run_held_out_evaluation.py` across the full 739-document held-out test set.

```

{
  "n_documents_attempted": 739,
  "n_documents_succeeded": 739,
  "n_documents_errored": 0,
  "run_time_seconds": 880,
  "corpus_level_cer_wer": {
    "ocr_only_cer": 0.0492,
    "ocr_only_wer": 0.056,
    "final_stitched_cer": 0.03,
    "final_stitched_wer": 0.0383
  },
  "reconstruction_accuracy": {
    "n_reconstructed_words": 781,
    "top1_accuracy_overall": 0.4032,

```

```

    "n_content_words": 521,
    "top1_accuracy_content_words": 0.2802
  },
  "uncertainty_flagging_at_locked_threshold": {
    "threshold": 0.5442,
    "flag_rate": 0.4655,
    "error_recall": 0.6576,
    "precision": 0.843,
    "unflagged_error_rate": 0.3823
  },
  "provenance_counts": {
    "total_words": 15680,
    "ocr_provenance": 14899,
    "reconstructed_provenance": 781
  }
}

```

APPENDIX D — REPRODUCIBILITY MANIFEST

- Corpus file: sentence_corpus.jsonl
- Test-split record count (verified at run time via SHA-256 hash check): 739
- Fine-tuned model: roberta-finetuned-final/ (RoBERTa-base, fine-tuned on masked-word reconstruction; hosted externally due to file size — see project repository README)
- Locked confidence threshold: 0.5442 (selected on validation set only, Step 4)
- Pinned library versions: transformers==4.44.2, tokenizers==0.19.1 (Section 5.5.1)
- Damage generator seed policy: reproducible, per-document seeded random tear generation (torn_regions.py, word_targeted_masks.py)
- Evaluation entry point: run_held_out_evaluation.py, producing held_out_manifest.json plus one JSON result file per document (results/held_out_documents/test_0000.json ... test_0738.json)
- Schema file: document_schema.json (JSON Schema draft 2020-12; reproduced in full in Appendix A)
- Repository: <https://github.com/aimlin9/document-recovery-ml-internship>
- Author LinkedIn: <https://www.linkedin.com/in/ramsey-opoku-gyimah-79a4b72aa>

APPENDIX E — HELD-OUT MANIFEST AND METRIC COMPUTATION

From run_held_out_evaluation.py: the manifest-writing function that ties every reported Step 5 figure to a file hash and a record count taken at run time (Section 7.3), together with the three threshold-based metric functions used to compute Table 2's uncertainty-flagging results.

```

"""
Step 5 batch validation: run the full image-based integration pipeline
(render -> targeted tear -> OCR -> damage classification -> reconstruction
-> confidence/uncertain flag -> schema-validated JSON) across every
held-out test sentence, and produce an aggregate report.

```

DENOMINATOR / MANIFEST: the held-out set used here is whatever sentence_corpus.jsonl currently contains under split == "test" on this machine, right now - not a remembered number from earlier in the project. This script hashes that file and records the exact count into held_out_manifest.json, so the final report cites a file-verified number. Do not hand-edit that count anywhere else.

METHODOLOGY NOTE (read before comparing numbers to Step 4): Step 4's 1,478-example benchmark was pure text - two words masked per sentence, no image, no OCR, no tear. This script instead renders each held-out sentence to an image and deliberately damages exactly ONE randomly chosen word per sentence using word_targeted_masks.build_targeted_mask, so that every document contributes exactly one reconstruction attempt and none are wasted on a random tear that happens to miss all text. That makes N here equal to the number of held-out SENTENCES, not 1,478 - a different, smaller denominator than Step 4's, by design.

```
def load_held_out_sentences():
    with open(CORPUS_PATH, "rb") as f:
        raw = f.read()
        file_hash = hashlib.sha256(raw).hexdigest()
        records = [json.loads(line) for line in raw.decode("utf-8").splitlines()]
        test_records = [r for r in records if r["split"] == "test"]

    manifest = {
        "corpus_file": CORPUS_PATH,
        "corpus_file_sha256": file_hash,
        "total_records_in_file": len(records),
        "held_out_test_count": len(test_records),
        "note": "This count is read from the file at run time, not hardcoded. "
               "If this number ever differs from a previously reported count, "
               "trust this one and flag the discrepancy explicitly rather than "
               "silently averaging or picking whichever number is convenient.",
    }
    with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
        json.dump(manifest, f, indent=2)
    print(f"Manifest written to {MANIFEST_PATH}: {len(test_records)} held-out "
          "sentences "
          f"(corpus sha256 {file_hash[:16]})")
    return test_records

def error_recall_at_threshold(results, threshold):
    wrong = [r for r in results if not r["is_correct"]]
    if not wrong:
        return None
    return sum(1 for r in wrong if r["confidence"] <= threshold) / len(wrong)

def precision_at_threshold(results, threshold):
    flagged = [r for r in results if r["confidence"] <= threshold]
    if not flagged:
```

```

        return None
    return sum(1 for r in flagged if not r["is_correct"]) / len(flagged)

def unflagged_error_rate(results, threshold):
    unflagged = [r for r in results if r["confidence"] > threshold]
    if not unflagged:
        return None
    return sum(1 for r in unflagged if not r["is_correct"]) / len(unflagged)

```

APPENDIX F — GLOSSARY OF TERMS

- Provenance — the schema field recording whether a word's final text came from OCR ("ocr") or from the reconstruction model ("reconstructed"); the mechanism by which a reader can always tell an observation from a guess (Section 6, Section 9.3).
- Damage status — a per-word classification (intact, partial, missing) describing how much of a word's bounding box overlaps the known damage mask (Section 5.3).
- Uncertain flag — a boolean, set when a reconstructed word's confidence falls at or below the locked decision threshold, indicating the word should be reviewed by a human before being trusted (Section 5.5.3).
- Confidence (reconstruction) — the geometric mean of the per-subword-token probabilities the model assigned to its own chosen tokens while decoding a candidate word (Section 5.5.2).
- Corpus-level CER/WER — character/word error rate computed by pooling edit-distance counts across every document in the evaluation set before dividing, rather than averaging per-document rates (Section 5.7).
- Expected Calibration Error (ECE) — a single number summarizing, across confidence bins, how far a model's stated confidence departs from its observed accuracy (Section 5.7).
- Held-out test set — the 739-document set of images, each containing exactly one deliberately targeted damaged word, used for the Step 5 end-to-end evaluation and never used during fine-tuning (Section 5.1, Section 5.6.4).
- Text-only masked evaluation — the 1,478-example Step 4 evaluation set of masked sentences (no images involved), used to fine-tune and calibrate the reconstruction model before the image-based Step 5 evaluation (Section 5.5, Section 7.3).
- Data contract — this project's term for the JSON Schema (Appendix A) that formally specifies what every pipeline stage must produce, checked automatically rather than assumed (Section 5.6.1, Section 6).
- Generalization gap — the documented difference between the confidence layer's text-only calibration performance and its real, image-based evaluation performance (Section 9.1), treated in this paper as a primary finding rather than a defect.

APPENDIX G — NOTATION AND SYMBOL REFERENCE

Symbols used in the formal metric definitions of Section 5.7, collected here for reference.

- S, D, I — the number of substitutions, deletions, and insertions, respectively, in the minimum-edit-distance alignment between a hypothesis and a reference string.
- N — the length of the reference (in characters for CER, in whitespace-delimited words for WER).
- n — the number of subword tokens comprising a single reconstructed word, when computing geometric-mean confidence (Section 5.5.2).
- p_i — the model's assigned probability for its chosen token at decoding step i .
- b — an index over the ten equal-width confidence bins used to compute Expected Calibration Error.
- n_b, N — the number of predictions in bin b , and the total number of predictions across all bins, respectively (Section 5.7).
- $\text{acc}(b), \text{conf}(b)$ — the observed (empirical) accuracy and the average stated confidence within bin b , respectively.
- IoU — Intersection-over-Union, the ratio of the overlapping area to the union area of two bounding boxes, used to match OCR-detected words to their expected ground-truth positions (Appendix B.2).
- CI — confidence interval; this paper reports a 95% bootstrap confidence interval (10,000 resamples) around the Top-1 accuracy improvement attributed to fine-tuning (Section 5.5.3).